

6 : The Processor (Advanced)

Rajesh Panicker,
NUS

CG3207

Note :

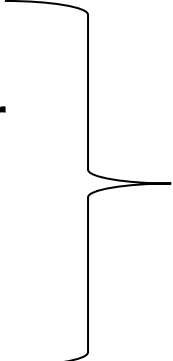
- Slides not covered in detail in the class are left as a self-learning exercise

Acknowledgement :

- Text by Harris and Harris and companion materials
- Text by Patterson & Hennessy and companion materials by Mary Jane Irwin, PSU



How to Extract **Even More** Performance?

- Branch Prediction
 - Deep Pipelining
 - Micro-operations
 - Superscalar Processor
 - Out of Order Processor
 - Register Renaming
 - VLIW Processor
 - Hardware Multithreading
 - Multicore / processor Systems
 - SIMD/Vector/SIMT
 - Systolic Arrays
- Multiple Issue Processors
- 

Control Hazards : Static Branch Prediction

- Resolve branch hazards by assuming a given outcome and proceeding without waiting to see the actual branch outcome
- 1. Predict not taken – always predict branches will not be taken, continue to fetch from the sequential instruction stream, only when branch *is* taken does the pipeline stall
 - If taken, flush instructions after the branch (earlier stage of execution in the pipeline)
 - Ensure that those flushed instructions haven't changed the machine state – stall if required
 - Isn't a problem if the branch is committed in E-stage, but is a problem if in W (memory could be modified) stage
 - Restart the pipeline at the branch destination
 - We had implicitly implemented this scheme in the earlier design!
- 2. Predict taken – predict branches will always be taken
 - *Where* will the branch be taken to? – need some mechanism to predict that too! (this is generally easier than predicting *whether* a branch will be taken)

Static Branch Prediction

- Doesn't use any runtime information
- Used in simpler processors, or as a fallback when there isn't enough info for dynamic prediction, e.g., first time a segment of code is executed
- "Predict not taken" works well for "top of the loop" branching structures
 - `for` loop, `while` loop
- "Predict not taken" doesn't work well for "bottom of the loop" branching structures
 - `do-while` loop
- Other static prediction schemes exist, such as
 - The compiler inserts a 'hint' bit (used in Pentium 4)
 - Predict that backward branches are taken, forward branches are not taken (BTFN) – can be done easily by looking at the sign bit of the offset

```
Loop: beq s1, s2, Out
      1st loop instr
      .
      .
      .
      last loop instr
      j Loop
Out:   fall out instr
```

```
Loop: 1st loop instr
      2nd loop instr
      .
      .
      .
      last loop instr
      bne s1, s2, Loop
      fall out instr
```

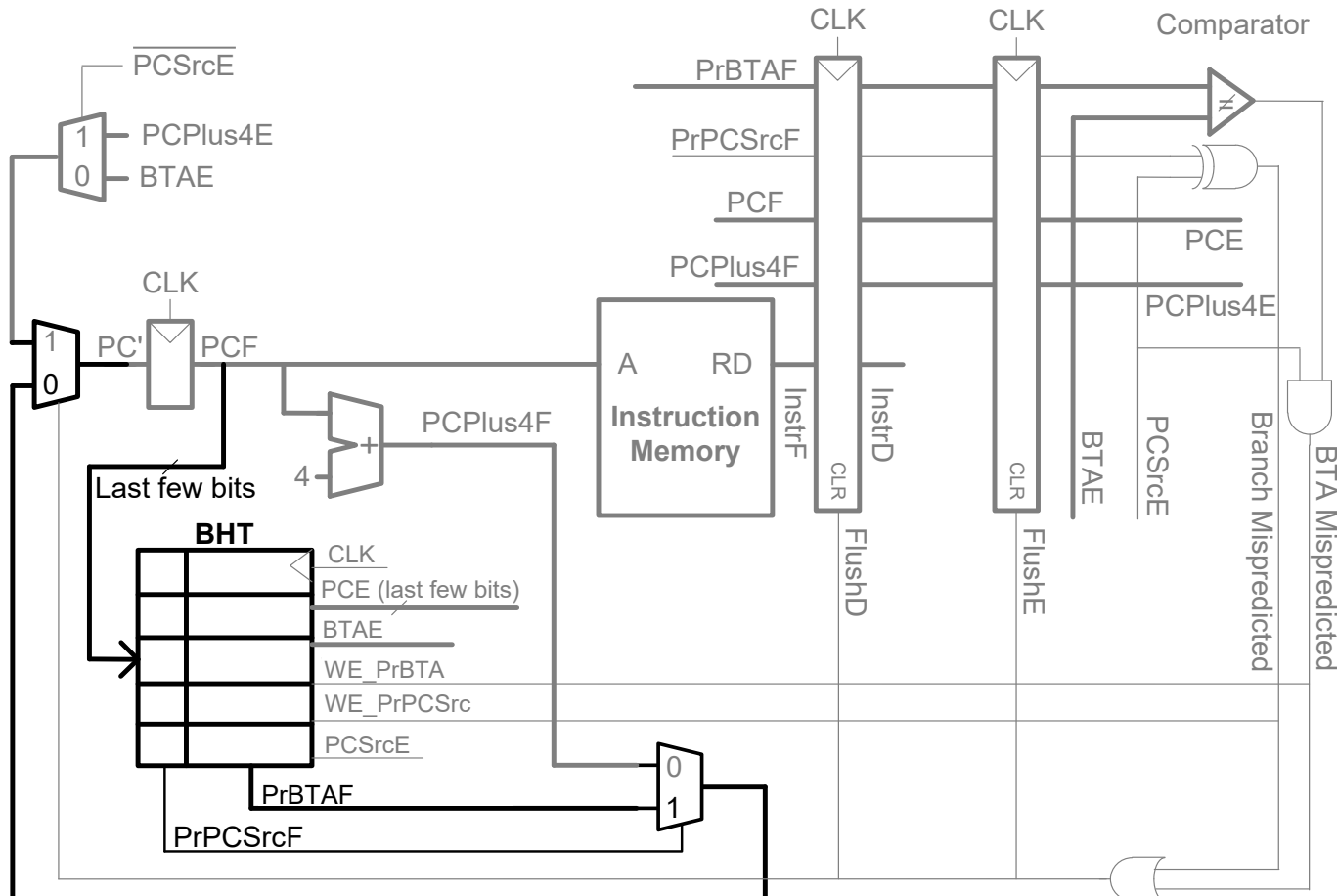


Dynamic Branch Prediction

- As the branch penalty increases (i.e., for deeper pipelines), a simple static prediction scheme will hurt performance. With more hardware, it is possible to try to predict branch behavior dynamically during program execution (i.e., using *run-time* information)
- A branch prediction buffer (aka branch history table (BHT)) in the F stage addressed/indexed by a few lower bits of the PC, contains bit(s) that tells *whether* the branch was taken the last time it was executed
- Similarly, a branch target buffer (BTB) caches the BTA computed when the branch was taken last time. It gives a prediction of *where* the branch is taken to. It is often incorporated into the BHT itself
- If the prediction is perfect, stalls can be avoided entirely

Branch History Table

Not considering jalr here. Otherwise, BTAE expression will involve RD1E



PrBTA = Predicted BTA when the instruction is in F stage

PrPCSrcF = Predicted PCSrc when the instruction is in F stage = Branch Prediction

BTA = $\text{PCE} + \text{ExtImmE}$ = Actual BTA when the instruction is in E stage

PCSrcE = Actual PCSrc when the instruction is in E stage = Branch Outcome

When Prediction Goes Wrong

- Branch / BTA prediction could be wrong
 - May be a wrong prediction for this branch this iteration
 - May be from a different instruction (possibly not even branch) with the same lower order PC bits
 - In any case, we cannot compromise on *correctness*, though we can compromise on *performance*
- If the prediction is wrong
 - Flush the incorrect instruction(s) in pipeline
 - Restart the pipeline with the correct PC value
 - Update the prediction bit(s) and / or predicted branch target
- A 4096 entry (12-bit) BHT varies from 1% misprediction (nasa7, tomcatv) to 18% (eqntott)
 - 4096 entry table is about as good as an infinite table, but 4096 entries would need a lot of hardware. Also, bigger tables -> slower

1-bit Prediction Accuracy

- A 1-bit predictor will be incorrect twice
 - Assume predict_bit = 0 to start (indicating branch not taken) and loop control is at the bottom of the loop code

1. First time through the loop, the predictor mispredicts the branch since the branch is taken back to the top of the loop; invert prediction bit (predict_bit = 1)
2. As long as branch is taken (looping), prediction is correct
3. Exiting the loop, the predictor again mispredicts the branch since this time the branch is not taken falling out of the loop; invert prediction bit (predict_bit = 0)

```
mv    s1, #10
Loop: 1st loop instr
      2nd loop instr
      .
      .
      .
      last loop instr
      sub    s1, #1
      bnez   Loop
      fall out instr
```

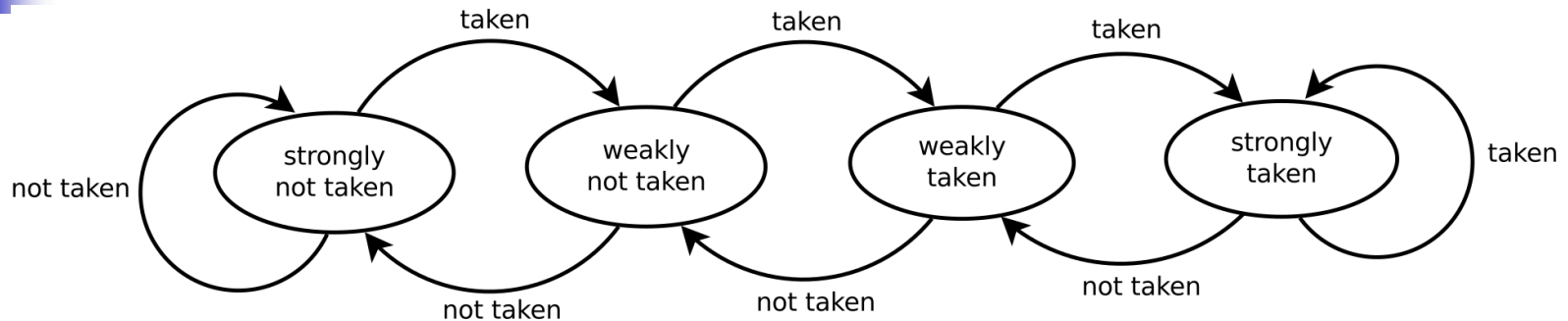
Prediction correct?

First : NYYYYYYYYYN

Subsequent : NYYYYYYYYYN

- For 10 times through the loop we have a 80% prediction accuracy for a branch that is taken 90% of the time

2-bit Predictors



- A 2-bit scheme can give 90% accuracy since a prediction must be wrong twice before the prediction bit is changed
- BHT stores the FSM state.
- This approach is usually called a saturating counter
 - BHT uses temporal correlation
- There are two-level predictors that make use of spatial correlation – a preferred path of execution

```

mv    s1, #10
Loop: 1st loop instr
      2nd loop instr
      .
      .
      .
      last loop instr
      sub    s1, #1
      bnez Loop
      fall out instr
  
```

Prediction correct?

First : NNNYYYYYYN
(starting state : Strongly Not Taken)

Subsequent : YYYYYYYYYN
(starting state : Weakley Taken)



Types of Parallelism

- Key to performance improvement – exploiting parallelism
- Instruction-level parallelism (ILP) of a program – a measure of the average number of instructions in a program that a processor *might* be able to execute at the same time
 - Mostly determined/limited by the dependencies between instructions. Usually < 3
 - Exploited via pipelining, multiple issue processors
- Data parallelism
 - Ability of different data points to be operated upon in parallel
 - Exploited by most hardware accelerators, GPUs
- Thread-level/task parallelism
 - Different programs/parts of a program being able to execute in parallel (control parallelism)
 - Exploited by hardware multithreading/multicore/multiprocessors
- To exploit the above, need additional hardware (machine parallelism)



Deep Pipelining

- 10-20 stages typical. Diminishing returns with additional stages
 - Sequencing overhead – Pipeline register setup time and (clock to Q) propagation delay becomes significant
 - The share of actual logic in the clock period decreases – increased latency
 - Pipeline hazards – increased possibility of hazards
 - Requiring more hazard and forwarding logic
 - Delay caused by forwarding logic (such as muxes) becomes significant too, diminishing returns further
 - Increased power and cost from hazard/forwarding logic
 - Higher misprediction penalty – need to flush more instructions
-> better branch prediction required -> even more hardware
 - Example : Intel Pentium 4 (Netburst)
 - Upto 31 pipeline stages, 20 cycles misprediction penalty

Micro-operations

- Decompose more complex instructions into a series of simple instructions called *micro-operations* (*micro-ops* or μ -ops)
- At run-time, complex instructions are decoded into one or more micro-ops
- Used heavily in CISC (complex instruction set computer) architectures (e.g., x86)
- Used for some ARM instructions, for example:

Complex Op

LDR R1, [R2], #4

Micro-op Sequence

LDR R1, [R2]

ADD R2, R2, #4

Without u-ops, would need 2nd write port on the register file

Contrast this with *micocodes* which are rigid sequences of steps to execute a complex instruction, can't be reordered or pipelined

Micro-operations ...

- Decoding done by hardware - the programmer/compiler doesn't need to know what is under the hood. The micro-operations need not even be valid instructions in the ISA
 - Micro-ops are generally wider and regular in format than usual instructions, and may be different in diff. microarchitectures
 - Can be reordered by the processor (superscalar) hardware
- Allow for dense code (fewer memory accesses)
- Yet preserve simplicity of RISC hardware
 - Allows a separation between front-end (fetch and decode) and back-end (execution and commit/writeback units)
- Modern x86 processors include a micro-op cache (execution trace cache) for decoded micro-ops
 - An instruction present in the cache need not be decoded again!

Micro-operations vs pseudo-instructions

The assembler splits pseudo-instructions (which are **not valid instructions** in the ISA) into **valid instructions** within the ISA. The instructions to which the pseudo-instruction is split into is visible to the programmer.

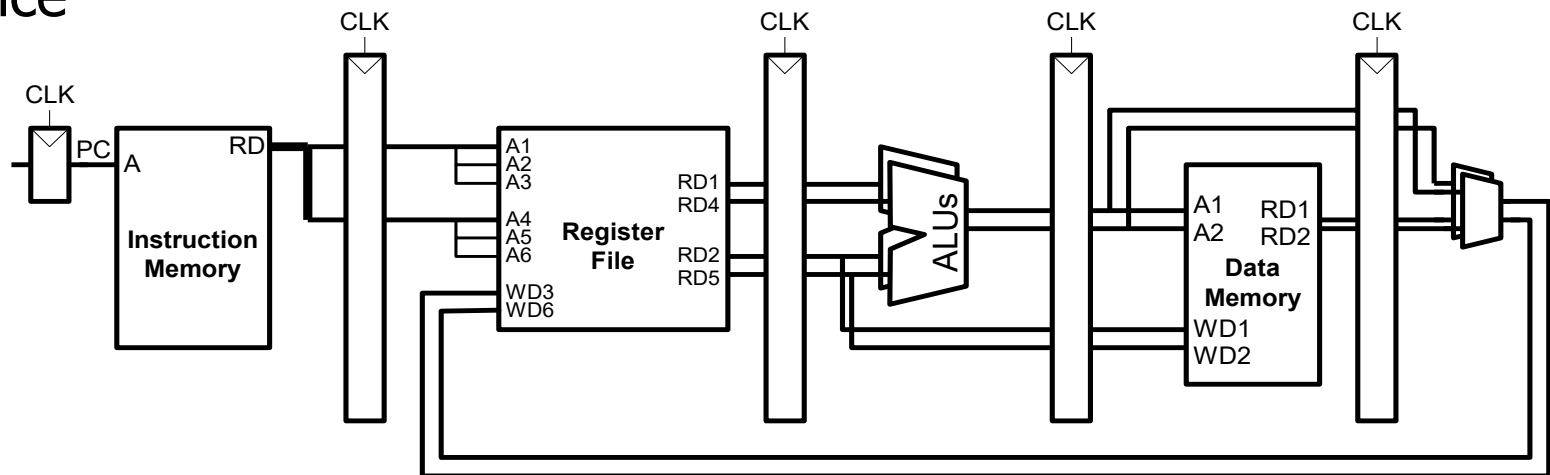
In case of micro-operations, the hardware splits the complex instructions (which are **valid instructions** in the ISA) into simple instructions/operations which are **not necessarily within the ISA**. Micro-operations are invisible to the programmer.

Macro-op Fusion

- Almost the opposite of micro-operations – two instructions that are nearby (usually consecutive) are fused together by the hardware and executed as a single instruction
- Reduces the number of instructions actually executed by the backend. Could also reduce power
- Example: In ARM, CMP sets flags, which are then used by a following branch. They could be fused by the front-end and executed by the back-end as a RISC-V style compare+branch
- mul+mulh can be executed as a single long multiply internally
 - mul x4, x2, x3 + mulh x5, x2, x3 could be fused as something like SMULL x5, x4, x2, x3
 - But this will need 2 write ports for register file
 - This can be split into micro-operations SMULL x5, tmp, x2, x3 + add x4, x0, tmp where tmp is a register not in the register file!

Multiple-Issue Processors

- Fetch (and execute) more than one instructions at one time (expand every pipeline stage to accommodate multiple instructions) – multiple-issue
- Multiple copies of datapath execute multiple instructions at once
 - The instruction execution rate, CPI, will be less than 1, so instead we use IPC: instructions per clock cycle
- Dependencies make it tricky to issue multiple instructions at once





Multiple-Issue Processor Styles

- Dynamic multiple-issue processors (aka superscalar)
 - Decisions on which instructions to execute simultaneously (usually in the range of 2 to 8) are being made dynamically (at run time by the hardware)
 - Most modern processors are superscalar

- Static multiple-issue processors (aka Very Large Instruction Word - VLIW)
 - Decisions on which instructions to execute simultaneously are being made statically (at compile time by the compiler)

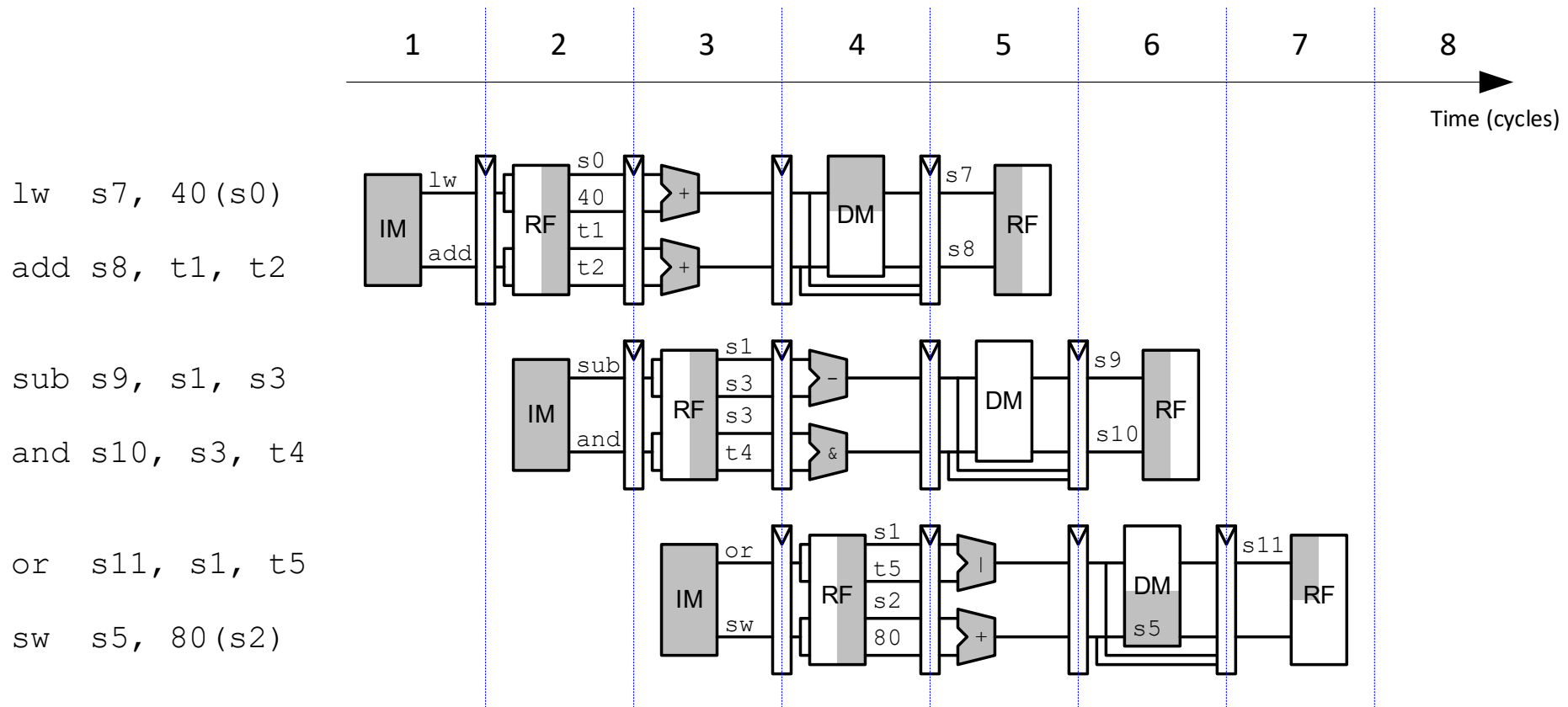


Multiple-Issue Datapath Responsibilities

- Must handle, with a combination of hardware and software fixes, the fundamental limitations of
 - How many instructions to issue in one clock cycle – issue slots
 - Storage (data) dependencies – aka data hazards
 - Limitation more severe in a SS/VLIW processor due to (usually) low ILP
 - Procedural dependencies – aka control hazards
 - Ditto, but even more severe
 - Use dynamic branch prediction to help resolve the ILP issue
 - Resource conflicts – aka structural hazards
 - A SS/VLIW processor has a much larger number of potential resource conflicts
 - Functional units may have to arbitrate for result buses and register-file write ports
 - Resource conflicts can be eliminated by duplicating the resource or by pipelining the resource

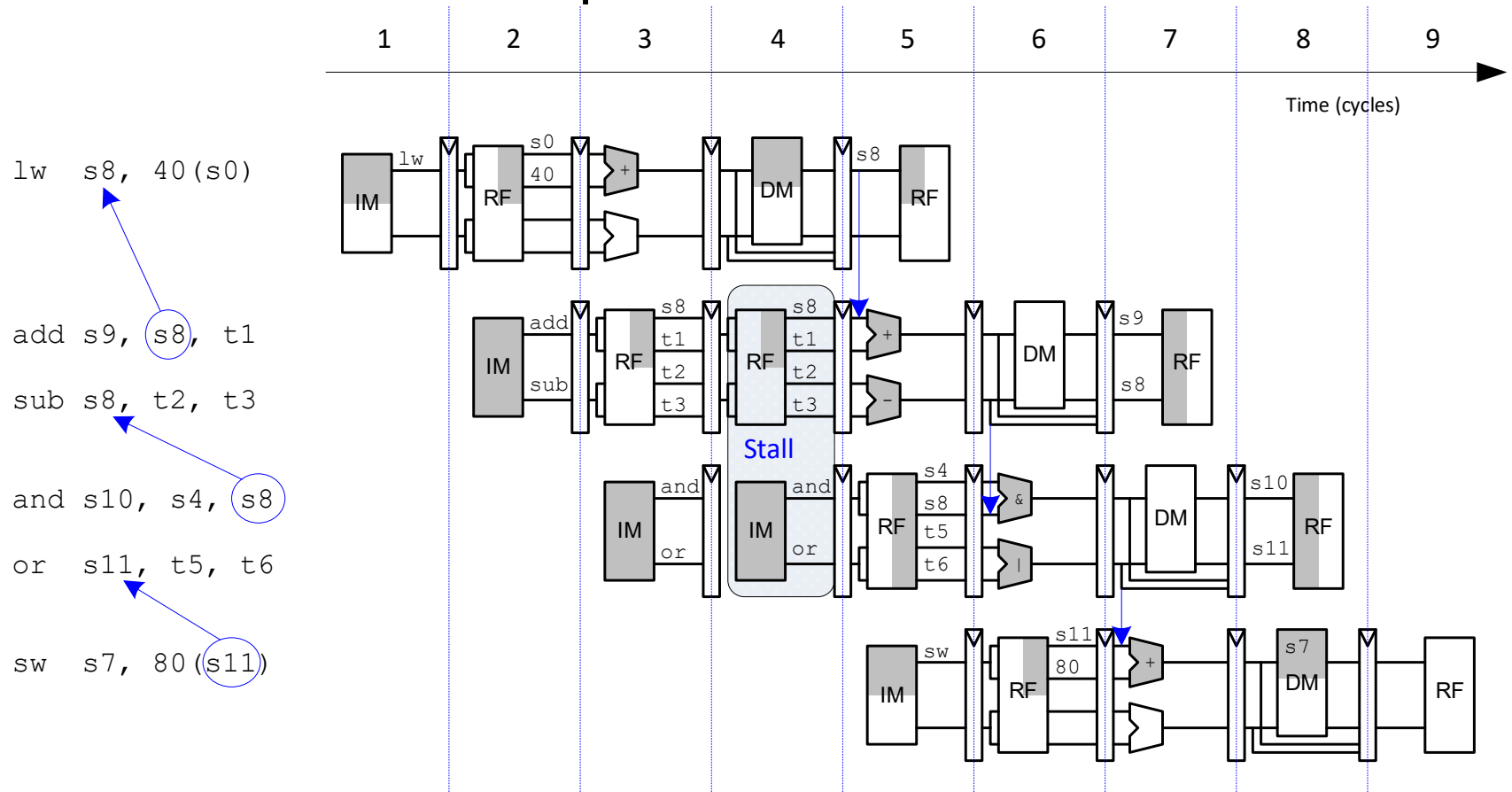
Superscalar – Ideal Case

- Ideal IPC = 2
- Actual IPC = 2



Superscalar – Practical Case

- Ideal IPC = 2
- Actual IPC = 1.2. Dependencies limit the achievable IPC



Out of Order Processor

- Looks ahead across multiple instructions
- Issues as many instructions as possible at once
- Executes instructions out of order (as long as no dependencies)
- Dependencies:
 - RAW (read after write, a.k.a true data dependency)
 - WAR (write after read, a.k.a antidependency)
 - WAW (write after write, a.k.a output dependency)

} storage conflicts

L1 : $\text{R3} = \text{R3} * \text{R5}$

L2 : $\text{R4} = \text{R3} + 1$

L3 : $\text{R3} = \text{R5} + 1$

Antidependency

True data dependency

Output dependency

If L3 executes before L1 / L2, L1 / L2 will use the value of R3 written by L3

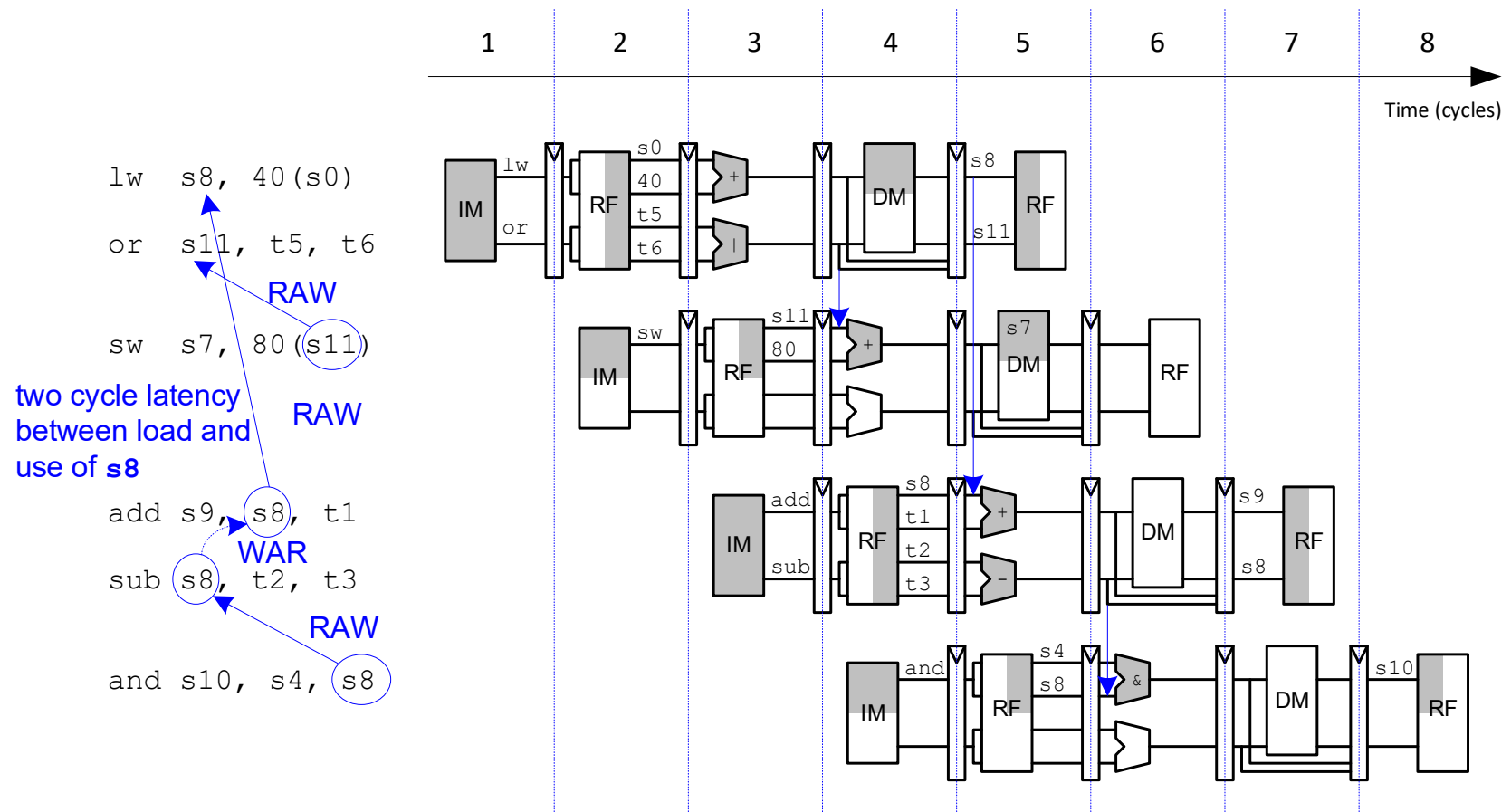
If L3 executes before L1, statements after L3 will use the value of R3 written by L1

Out of order Example

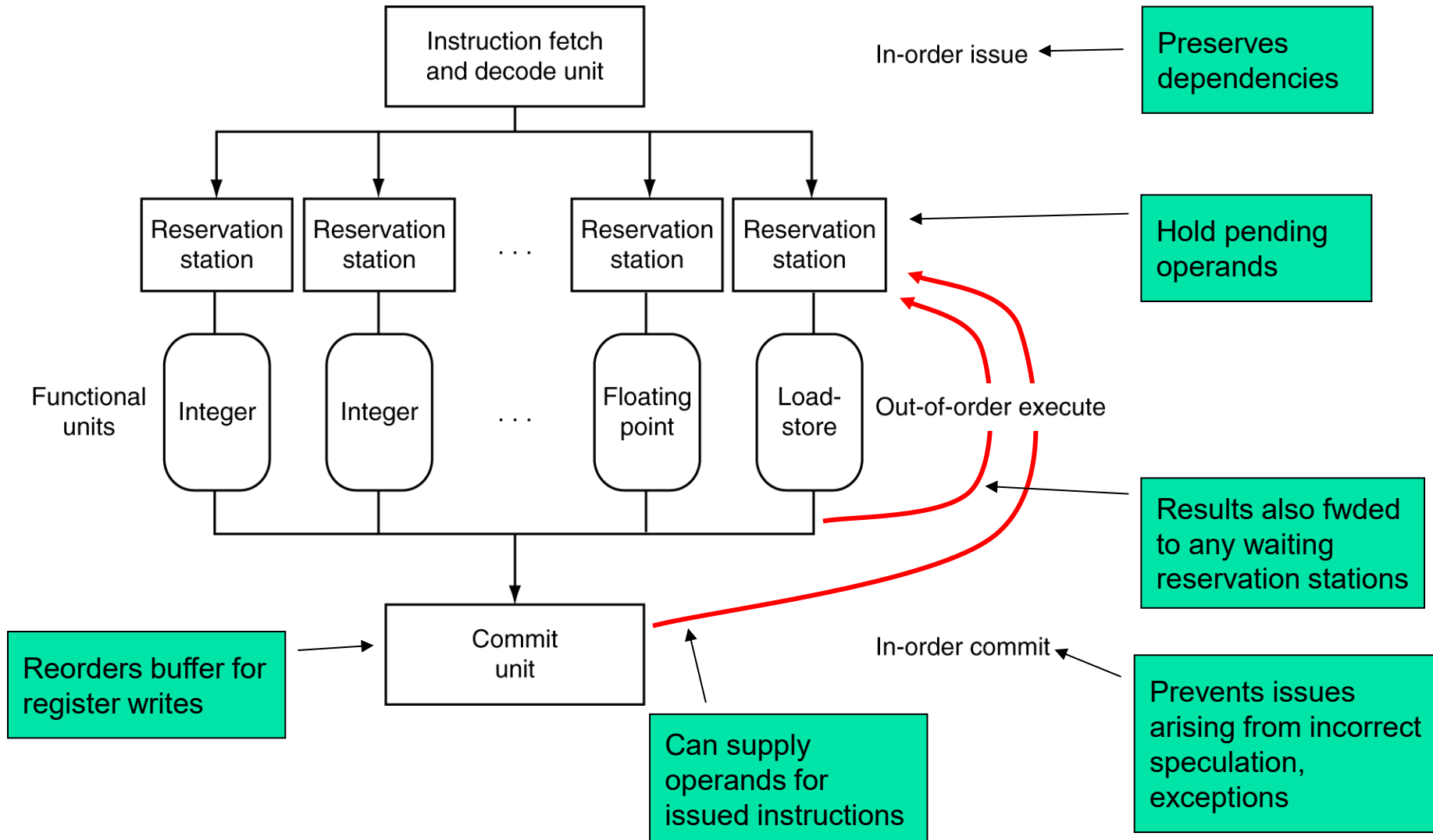
Ideal IPC : 2

Actual IPC : $6/4 = 1.5$

```
lw  s8, 40(s0)
add s9, s8, t1
sub s8, t2, t3
and s10, s4, s8
or  s11, t5, t6
sw  s7, 80(s11)
```



Out of order Execution





In-Order vs Out-of-Order

- Instruction fetch and decode units are required to issue instructions in-order so that dependencies can be tracked
- The commit unit is required to write results to registers and memory in program fetch order so that
 - if exceptions occur the only registers updated will be those written by instructions before the one causing the exception
 - if branches are mispredicted, those instructions executed after the mispredicted branch don't change the machine state (i.e., we use the commit unit to correct incorrect speculation)
- Although the front end (fetch, decode, and issue) and back end (commit) of the pipeline run in-order, the FUs are free to initiate execution whenever the data they need is available – out-of-(program) order execution
 - Allowing out-of-order execution increases the amount of ILP

Storage Conflicts and Register Renaming

- Storage conflicts can be reduced (or eliminated) by increasing or duplicating the troublesome resource
 - Provide additional registers that are used to reestablish the correspondence between registers and values
 - Physical regs allocated dynamically by the hardware in SS processors
 - ISA registers used by the programmer just an abstraction! (logical)
- Register renaming – the processor renames the original register identifier in the instruction to a new register (one not in the ISA / visible register set)

$$\begin{aligned} \text{R3} &= \text{R3} * \text{R5} \\ \text{R4} &= \text{R3} + 1 \\ \text{R3} &= \text{R5} + 1 \end{aligned}$$



$$\begin{aligned} \text{R3b} &= \text{R3a} * \text{R5a} \\ \text{R4a} &= \text{R3b} + 1 \\ \text{R3c} &= \text{R5a} + 1 \end{aligned}$$

- The hardware that does renaming assigns a “replacement” register from a pool of free registers and releases it back to the pool when its value is superseded and there are no outstanding references to it
 - The micro-ops generated from an instruction could use renamed registers directly

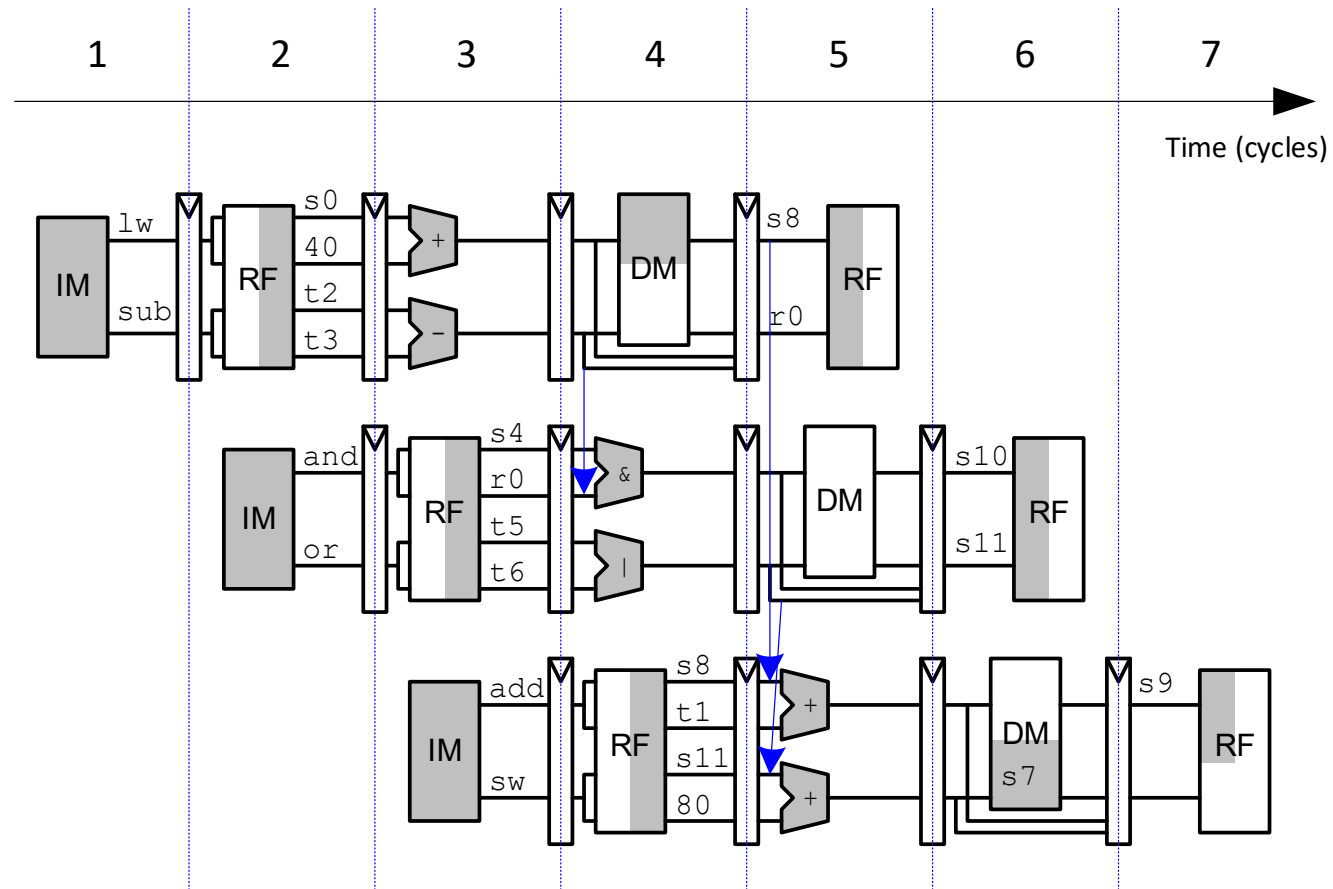
Register Renaming

Ideal IPC : 2

Actual IPC : $6/3 = 2$

```
lw  s8, 40(s0)
add s9, s8, t1
sub s8, t2, t3
and s10, s4, s8
or  s11, t5, t6
sw  s7, 80(s11)
```

`lw s8, 40(s0)`
`sub r0, t2, t3`
 2-cycle RAW RAW
`and s10, s4, r0`
`or s11, t5, t6`
 RAW
`add s9, s8, t1`
`sw s7, 80(s11)`





Speculative Execution

- Speculation is used to allow execution of future instructions that (may) depend on the speculated instruction
 - Speculate on the outcome of a conditional branch and execute instructions based on the prediction rather than the next instruction in the sequence (branch prediction)
 - Speculate that a store (for which we don't yet know the address) that precedes a load does not refer to the same address, allowing the load to be scheduled before the store (load speculation)
- Must have (hardware and/or software) mechanisms for
 - Checking to see if the guess was correct
 - Recovering from the effects of the instructions that were executed speculatively if the guess was incorrect
- Ignore and/or buffer exceptions created by speculatively executed instructions until it is clear that they should really occur
- Speculation can be a source of security vulnerabilities!



VLIW Processors

- The compiler packs groups of independent instructions into the bundle
 - The bundle can be thought of as one very long instruction. Hence the name
- The compiler uses loop unrolling to expose more ILP
- Compile time register renaming (using visible register set) used to solve storage dependencies
- While superscalars use dynamic prediction, VLIW's primarily depend on the compiler for branch prediction
 - Compiler encodes prediction as a hint bit in the branch instruction format
 - Loop unrolling reduces the number of conditional branches
 - Predication (conditional execution) eliminates if-then-else branch structures by replacing them with predicated instructions

VLIW Processor Example ...*

- Example : Intel Itanium and Itanium 2 for the IA-64 ISA – EPIC\$ (Explicit Parallel Instruction Computer)
 - 128-bit “bundles” containing three instructions, each 41-bits plus a 5-bit template field (specifying which FU each instruction needs)
 - Five functional units (IntALU, Mmedia, Dmem, FPALU, Branch), 128 integer registers, 128 floating point registers (82 bits long to preserve precision for intermediate results), ...
 - Extensive support for speculation and predication
 - Intel hoped it would take off in server market, and gradually percolate into PC/consumer market as 64-bit processing becomes popular
 - In contrast, AMD just extended the original 32-bit instruction set to 64-bit (x86_64 a.k.a AMD64) -> No need to recompile 32-bit code
 - “The Itanium approach...was supposed to be so terrific—until it turned out that the wished-for compilers were basically impossible to write” - Donald Knuth

\$ Not to be confused with AMD’s new Epyc x86_64 based server platform



VLIW vs Superscalar

- ⌚ Simpler hardware (potentially less power hungry)
- ⌚ Potentially more scalable - Allow more instructions per VLIW bundle and add more FUs
- ⌚ Programmer/compiler complexity and longer compilation times
- ⌚ Not all stalls are predicable - e.g., cache misses
- ⌚ Can't always schedule around branches - branch outcome is dynamically determined
- ⌚ Object (binary) code incompatibility – microarchitecture dependence
 - ⌚ In EPIC, correctness is guaranteed by hardware. The code will run *correctly* but not necessarily *optimally* on a different machine
- ⌚ Needs lots of program memory bandwidth
- ⌚ Code bloat - `NOPS` need to be inserted if the compiler can't find independent instructions; Loop unrolling to expose more ILP uses more program memory space
- Not popular for CPUs; used in GPUs (e.g., AMD GPUs), DSPs (TI TMS320 C6000 series), etc.



Does Multiple Issue Work?

- Yes, but not as much as we'd like
- Programs have real dependencies that limit ILP
- Some dependencies are hard to eliminate
- Some parallelism is hard to expose
 - Limited window size during instruction issue
- Memory delays and limited bandwidth
 - Hard to keep pipelines full
- Speculation can help if done well



Processes and Threads

- Process: program running on a computer
 - Multiple processes can run at once: e.g., surfing web, playing music, writing a report
- Thread: part of a program
 - Each process can have multiple threads: e.g., a word processor may have threads for typing, spell checking, printing etc., sharing the memory space (page table)
 - The operating system schedules threads
 - In a conventional processor, one thread runs at a time
 - In a multi-core / processor system, the number of threads running can be equal to the number of cores
 - In both the above cases, it appears to user that all threads running simultaneously



Threads in a Conventional Processor

- In *co-operative multitasking*, when one thread stalls (for example, waiting for I/O or similar events):
 - Architectural state of that thread stored
 - Architectural state of waiting thread loaded into processor and it runs
 - Called context switching
 - The next (ready) thread to be run determined by the OS scheduler
- In *preemptive multitasking*, the context switch can also be initiated through timer interrupts, without having to wait for the thread to stall



Hardware Multithreading

- Processor maintains multiple copies of architectural state (PC, registers, xPSR etc.)
 - OS sees it as multiple cores and schedules multiple threads
 - When one thread stalls for minor events such as cache misses, another runs immediately, without the OS initiating a context switch
 - This is called *coarse-grained multithreading*
- Does not increase instruction-level parallelism (ILP) of single thread, but increases throughput
 - Exploits thread-level parallelism
- Hardware threads are called 'harts' in RV terminology

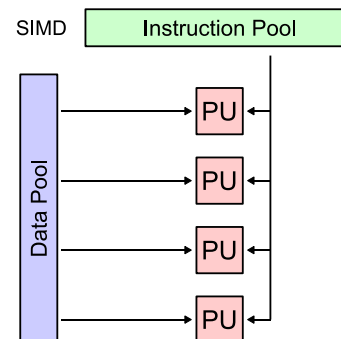
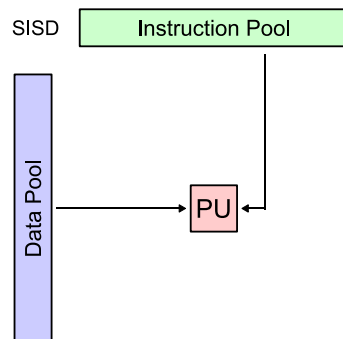


Hardware Multithreading

- Fine grained multithreading (barrel processor)
 - Executes one instruction from each thread in alternate clock cycles
 - Spaces instructions from a single thread by at least one instruction, thereby reducing dependencies and branch misprediction penalties as well as the hardware to deal with those
 - Used by GPUs heavily (much larger number of threads)
- Simultaneous multithreading (SMT a.k.a Hyperthreading)
 - If one thread can't keep all execution units busy, another thread can use them
 - Instructions from different threads execute at the same time, without duplication of functional units
 - Intel claims ~25% improvement in performance using less than 5% extra hardware
 - OS needs to be SMT-aware. Why?
- Coarse and fine-grained MT are temporal, SMT has both spatial and temporal aspects

Flynn's Taxonomy of Computer Arch.

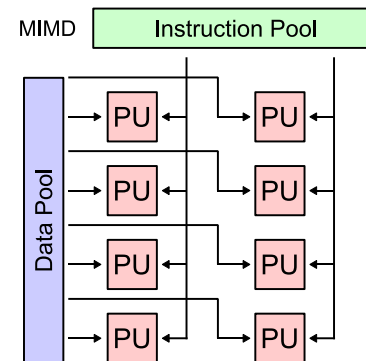
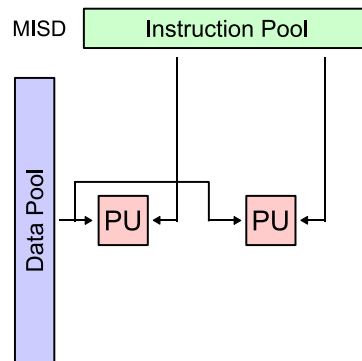
- Single instruction stream single data stream (SISD)
 - A sequential computer which exploits no parallelism in either the instruction or data stream
 - Traditional uniprocessor machines
- Single instruction stream, multiple data streams (SIMD)
 - A computer which exploits multiple data streams against a single instruction stream to perform operations which may be naturally parallelized
 - Exploits Data parallelism
 - For example, an array processor or graphics processing unit (GPU)



To Do :
Read up about
SSE Instructions

Flynn's Taxonomy of Computer Arch.

- Multiple instruction streams, single data stream (MISD)
 - Multiple instructions operate on one data stream
 - Uncommon architecture which is generally used for fault tolerance (ex : autopilot), systolic arrays (sort of MISD – we will see later)
- Multiple instruction streams, multiple data streams (MIMD)
 - Exploits Thread-level/task parallelism
 - Multiple autonomous processors simultaneously executing different instructions on different data
 - MIMD architectures include parallel / distributed systems, using either one shared memory space or a distributed memory space





Multiprocessor Systems

Motivation : Diminishing returns with increased clock frequency / cost

- The memory wall : the increasing gap between processor and memory speeds
- The ILP wall : the increasing difficulty of finding enough parallelism in a single instruction stream to keep a high-performance single-core processor busy
- The power wall : increasing power with operating frequency. It poses manufacturing, system design and deployment problems
- Multiprocessor systems exploit task / thread-level parallelism instead of trying to increase the performance of a single instruction stream



Loosely Coupled Multiprocessor Systems

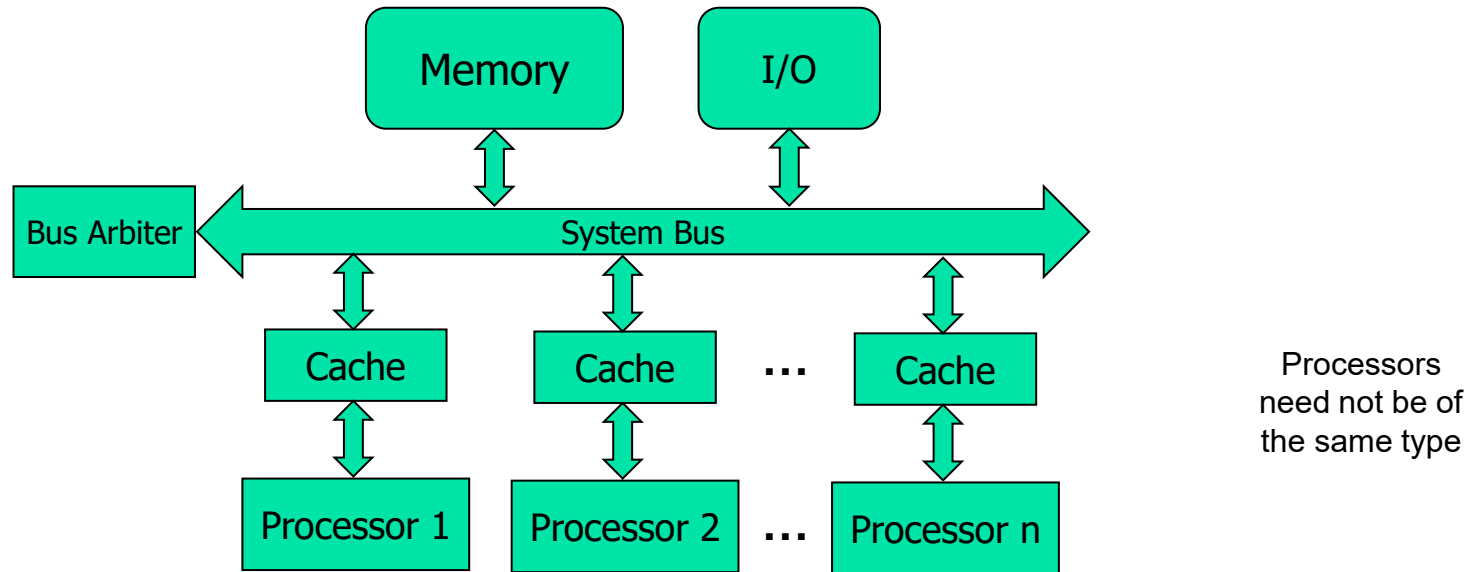
- Loosely coupled (*distributed*) multiprocessor systems – each node runs different OS instances and communicate by passing messages rather than through a shared memory
- Computer clusters - closely coupled - centralized job management & scheduling
 - Have nodes set to perform the same task, controlled and scheduled by software
 - Typically homogenous in hardware and software
 - Typically housed in the same building/geography, interconnected using a dedicated network, have shared resources
 - Often viewed as a single computer
 - e.g., Supercomputers
- Grid computers – very loosely coupled - distributed job management & scheduling
 - Have each node set to perform a different task to reach a common goal
 - Typically heterogenous and in hardware and autonomous in software
 - Geographically dispersed, diverse computers connected via internet
 - Sometimes just uses free cycles
 - e.g., seti@home, cryptocurrency mining pools



Tightly Coupled Multiprocessor Systems

- Tightly coupled (*parallel*) multiprocessor systems – usually mounted on the same mother board or within the same silicon die, communicates via shared memory and (usually) controlled by a single OS
 - Each processor executes different programs and works on different data
 - Each processor usually has one or more levels of private cache
 - They share many resources (higher level caches, memory, I/O device, interrupt system, etc.)
 - All processors connected using a system bus
- Multicore – multiprocessor system on a single die
 - Physical proximity allows the shared circuitry to operate at a much higher clock rate than what is possible if the signals have to travel off-chip

Tightly Coupled Multiprocessor Systems



- Shared devices (including the bus) need arbitration mechanisms if two processors attempt to use the same resource *simultaneously*
- Mutual exclusion mechanisms necessary to protect resources (or a range of memory) which should not be used in a *concurrent* manner
- Need mechanisms to ensure cache coherence across all levels
 - Bus snooping – the cache controller spies on every transaction on the bus to see if the data inside the cache is affected. Alternative to bus snooping : directory-based mechanisms



Symmetric vs Heterogeneous Multiprocessing

- Symmetric multiprocessing (SMP) systems are systems with two or more identical cores
 - Ex : Intel Core i3, i5 (prior to Alder Lake) etc.
- Heterogeneous multiprocessing (HMP) refers to systems that use more than one kind of processor or cores
 - Uses dissimilar processors or coprocessors - cores could be of totally different ISAs or different microarchitectures but same ISA
 - Usually incorporating specialized processing capabilities to handle particular tasks
 - Some coprocessors might not be ISA-based
 - Coprocessors are also called hardware accelerators
- Most modern systems have some level of heterogeneity
 - GPUs, Video codecs, cryptography coprocessors, programmable network processors, Neural/AI accelerators, DSPs, etc.
 - Most of these 'accelerators' perform SIMD-style processing



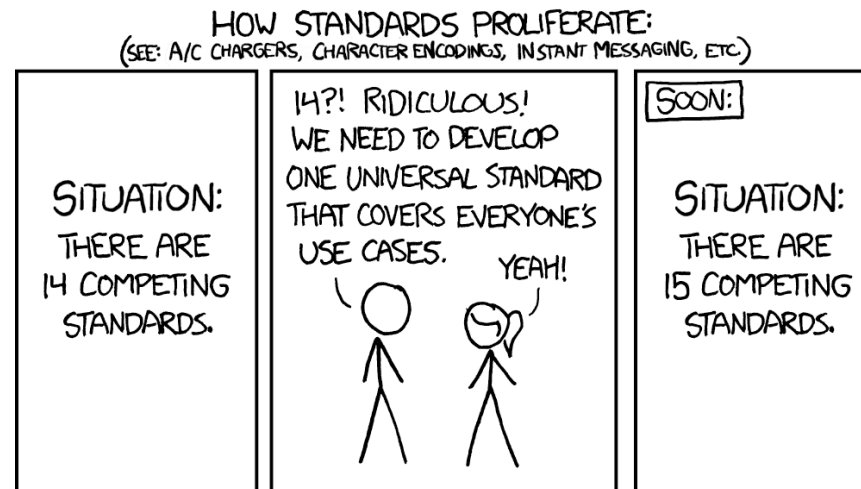
SMP Advantages and Disadvantages

- ⌚ Binary compatibility – the same binary can be used
- ⌚ Simplicity of system design, task scheduling and performance evaluation
- ⌚ Fault tolerance – if a particular core is known to be damaged, the system can still function at a reduced performance rather than failing altogether (also known as graceful degradation)
- ⌚ Higher power, cost for the same performance compared to HMP
 - Power issues mitigated to some extent by scaling down the voltage and/or frequency of cores based on load - Dynamic voltage and frequency scaling (DVFS)
 - In modern computers, some cores can be turned off altogether if the computational demand of the application(s) is low

HMP Advantages and Disadvantages

- Higher performance for a given cost
- Lower power consumption for a given performance
- Poses difficulty in design, debugging, performance evaluation, task scheduling. Various frameworks to manage this include

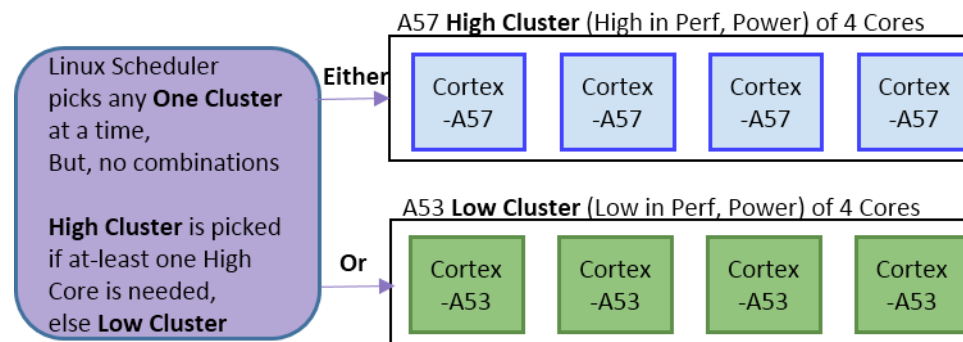
- OpenCL
- CUDA
- OpenAcc
- OpenMP
- SYSCL
- HIP/ROCm
- oneAPI
- C++ AMP



- Need different binaries for different cores (if ISAs are diff.)
- Limited / no fault-tolerance
- Data format / endianness issues

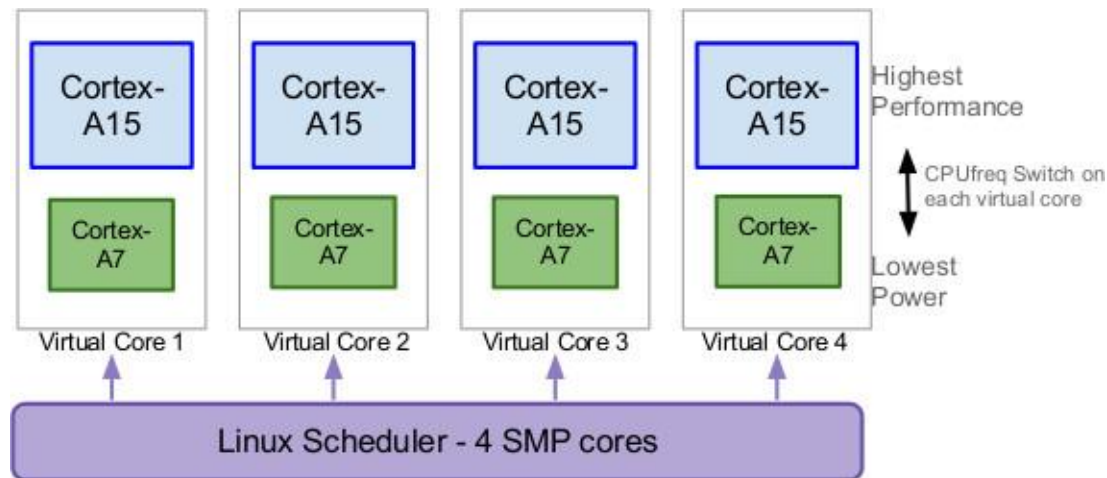
HMP : ARM big.LITTLE

- ARM big.LITTLE is a heterogeneous computing architecture coupling relatively battery-saving and slower processor cores (LITTLE) with relatively more powerful and power-hungry ones (big)
 - A multi-core processor that can adjust better to dynamic computing needs. ARM claims up to 75% savings in power
 - 3 Variants
- Clustered switching
 - Simplest implementation, arranging the processor into identically-sized clusters of 'big' or 'LITTLE' cores
 - Ex : Exynos 5 Octa 5410 (Samsung Galaxy S4), Apple A10 (iPhone 7)



HMP : ARM big.LITTLE

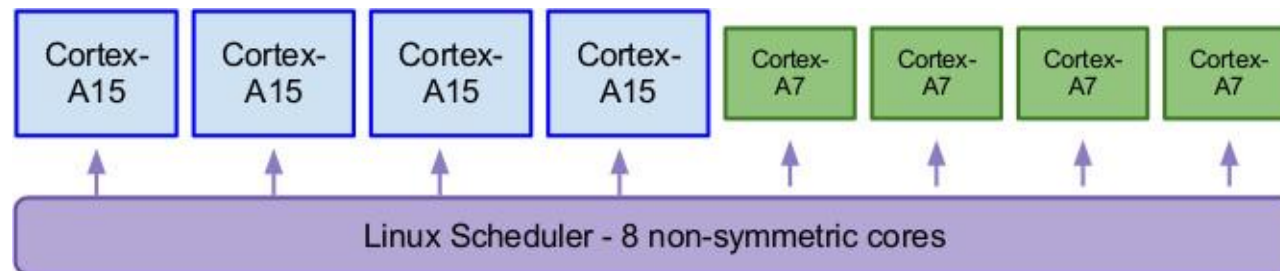
- In-kernel switcher (CPU migration)
 - CPU migration via the in-kernel switcher (IKS) involves pairing up a 'big' core with a 'LITTLE' core
 - Each pair operates as one virtual core, and only one real core is (fully) powered up and running at a time
 - Ex : Nvidia Tegra 3 SoC (Microsoft Surface [non-Pro], LG Optimus 4x HD,..)



HMP : ARM big.LITTLE

■ Global task scheduling

- The most powerful use model - enables the use of all physical cores at the same time (even more powerful with ARM DynamIQ – successor to big.LITTLE)
- Threads with high priority or computational intensity allocated to the 'big' cores while threads with lower priority or less computational intensity, such as background tasks, can be performed by the 'LITTLE' cores
- Ex : Exynos 5 Octa 5422 (Samsung Galaxy S5), Apple A12 (iPhone XS – 2 big and 4 LITTLE cores)



- Intel has implemented its version of this paradigm in Alder Lake processors released in early 2022 with 'Golden Cove' performance cores and 'Gracemont' efficient cores



HMP : Apple A14 SoC

- Apple A14 used in iPhone 12 has
 - 11.8 billion transistors, based on 5 nm fabrication process (TSMC)
 - first commercially available product based on 5 nm tech
 - Two high-performance cores called Firestorm and four energy-efficient cores called Icestorm running ARMv8 ISA
 - 4-core GPU
 - 16-core Neural Engine capable of performing 11 trillion operations per second
 - Machine learning matrix multiplication accelerators (AMX blocks)
 - Image processor for computational photography
 - A motion co-processor (M14) which reads accelerometer, gyroscope, compass, and barometer even if the device is asleep, and applications can retrieve data when the device is woken up
 - Video encoders/decoders,

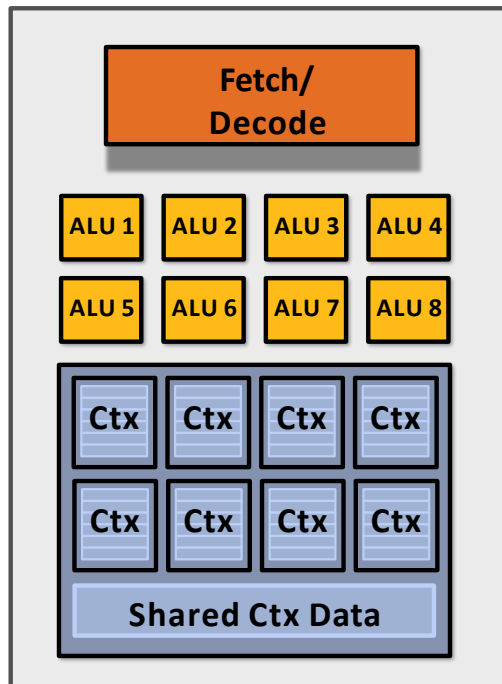


HMP with CPUs+FPGA

- Pairing FPGAs with CPUs is also common for custom hardware acceleration applications
 - Compute using FPGAs is called adaptive or reconfigurable computing
 - E.g., Microsoft Bing uses FPGA acceleration
- Intel/Altera Cyclone SoC series has FPGA+ARM Cortex processors
- Xilinx/AMD Zynq series has FPGA+ARM Cortex processors
- Some Intel Xeon CPUs have integrated FPGAs
- AMD has/had plans to integrate FPGAs into EPYC processors
- Many FPGAs are available as PCIe cards
 - Can be used as accelerators programmed/accessed through OpenCL framework
 - E.g., Xilinx/AMD Alveo, Versal
 - FPGA instances are offered by many cloud services such as Amazon (EC2 F1 Instances), Alibaba Cloud, etc

SIMD/Vector Processing

- Amortize cost/complexity of managing an instruction stream across many ALUs/PEs (Processing Elements) by exploiting data parallelism: SIMD processing using *Packed SIMD* or *Vector / Array* instructions
- Option 1: Processes n (say, 8) fragments using vector ops on vector registers
- Done using explicit SIMD/vector instructions in the ISA – x86 AVX / AVX-512, ARM NEON / SVE2 (AVX-512 and SVE2 have predicated execution)
 - RV has both V (vector) and P (packed SIMD, not frozen yet) standard extensions



SIMD/Vector instructions

```

VEC8_mu1  vec_r3, vec_v0, cb0[0]
VEC8_madd vec_r3, vec_v1, cb0[1], vec_r3
VEC8_madd vec_r3, vec_v2, cb0[2], vec_r3
VEC8_clmp vec_r3, vec_r3, 1(0.0), 1(1.0)
VEC8_mu1  vec_o0, vec_r0, vec_r3
VEC8_mu1  vec_o1, vec_r1, vec_r3
VEC8_mu1  vec_o2, vec_r2, vec_r3
VEC8_mov  o3, 1(1.0)
    
```

Packed SIMD vs

Vector:

Packed SIMD

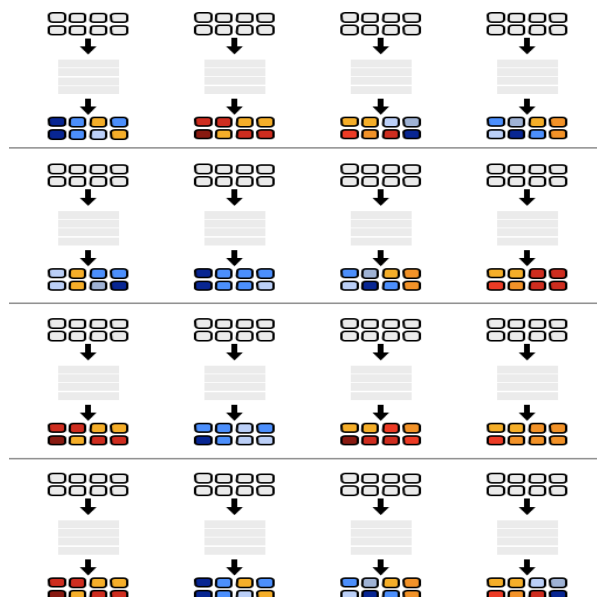
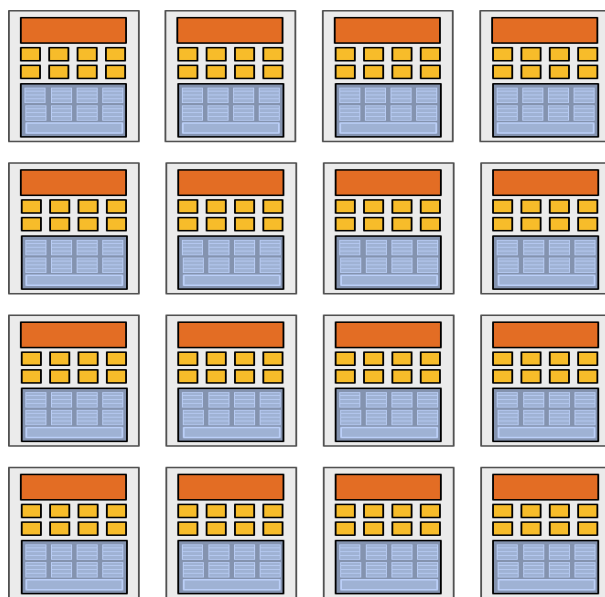
- Fixed length (though predication gives it a variable length flavour)
- On registers (as opposed to memories for vector)
- No reduction operations (e.g., sum of elements of a vector)

GPU/accelerator mantra: Spend less effort making decisions (control) and moving data (from host to device and back) and spend more time doing the same complex math on many fragments.

16 cores = 128 PEs
(CUDA cores), 16
simultaneous
instruction streams

GPU

- Option 2: Scalar instructions with implicit hardware vectorization
- Programmer specifies the instruction stream (kernel) to process one fragment (thread / work-item), as well as the size of fragment group assigned to a core (size of workgroup / thread block)
- HW determines instruction stream sharing across PEs - Nvidia ("SIMT" warps), AMD ("wavefronts") – the length of the vector formed by the hardware
- Uses predication to avoid branches. Interleaves processing of many warps/wavefronts on a single core to avoid stalls caused by high latency operations a.k.a fine-grained multithreading (trade-off latency for throughput)

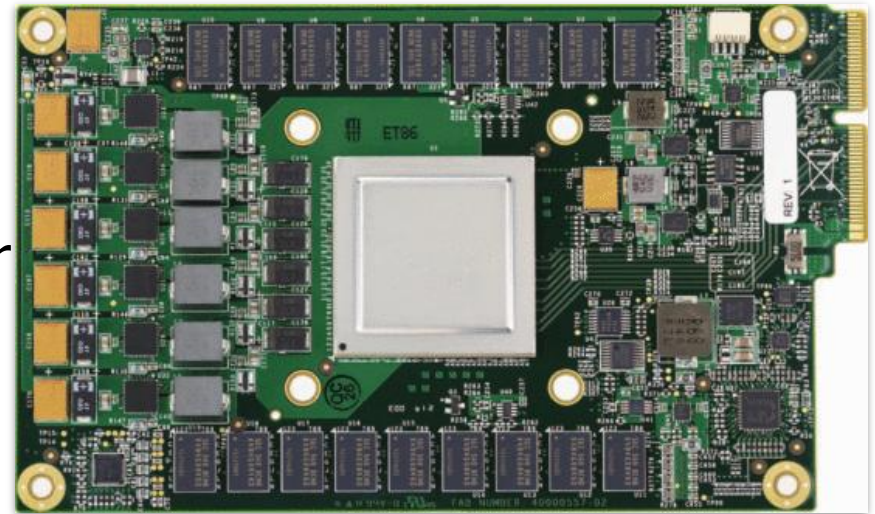


Scalar instructions (kernel)

```
mul  r3, v0, cb0[0]
madd r3, v1, cb0[1], r3
madd r3, v2, cb0[2], r3
clmp r3, r3, 1(0.0), 1(1.0)
mul  o0, r0, r3
mul  o1, r1, r3
mul  o2, r2, r3
mov  o3, 1(1.0)
```

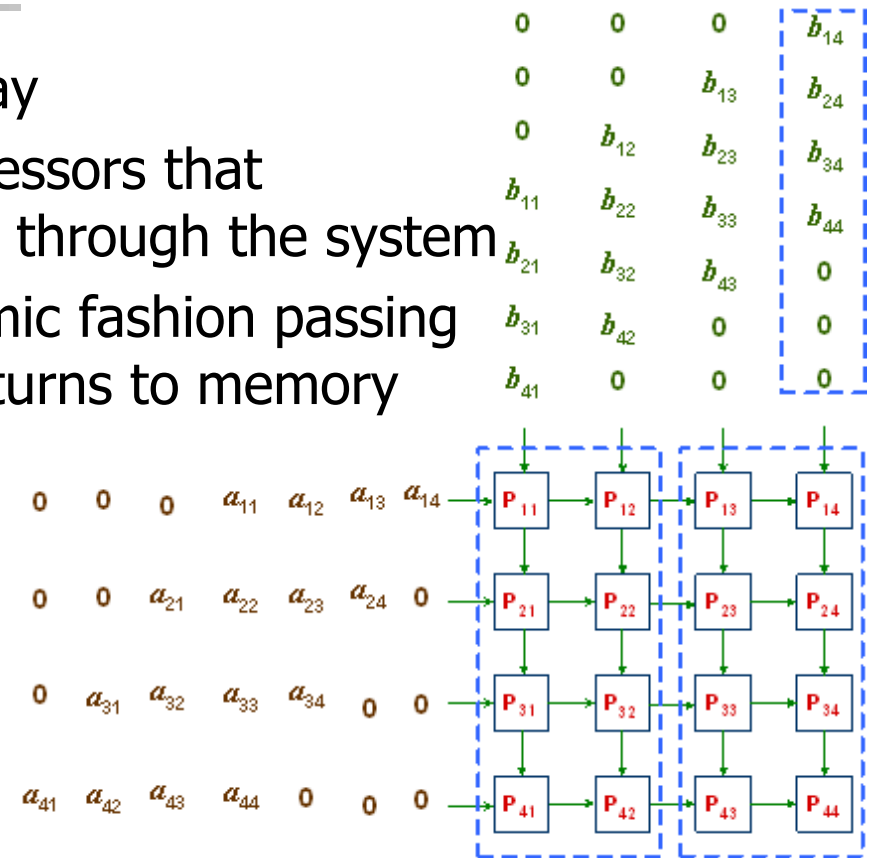
Google TPU (v1)

- 4 stage pipeline with CISC instruction set
- Placed on a PCIe port
- Interacts with CPU as a controller, where CPU sends instructions
- Used in data centers for heavy compute workloads
- The Matrix Unit: 65,536
(256x256)
- 8-bit multiply-accumulate units
- 700 MHz clock rate
- Peak: 92T operations/second
- 24 MiB of on-chip Unified Buffer
(activation memory)
- Two 2133MHz DDR3 DRAM
channels
- 8 GiB of off-chip weight DRAM
memory



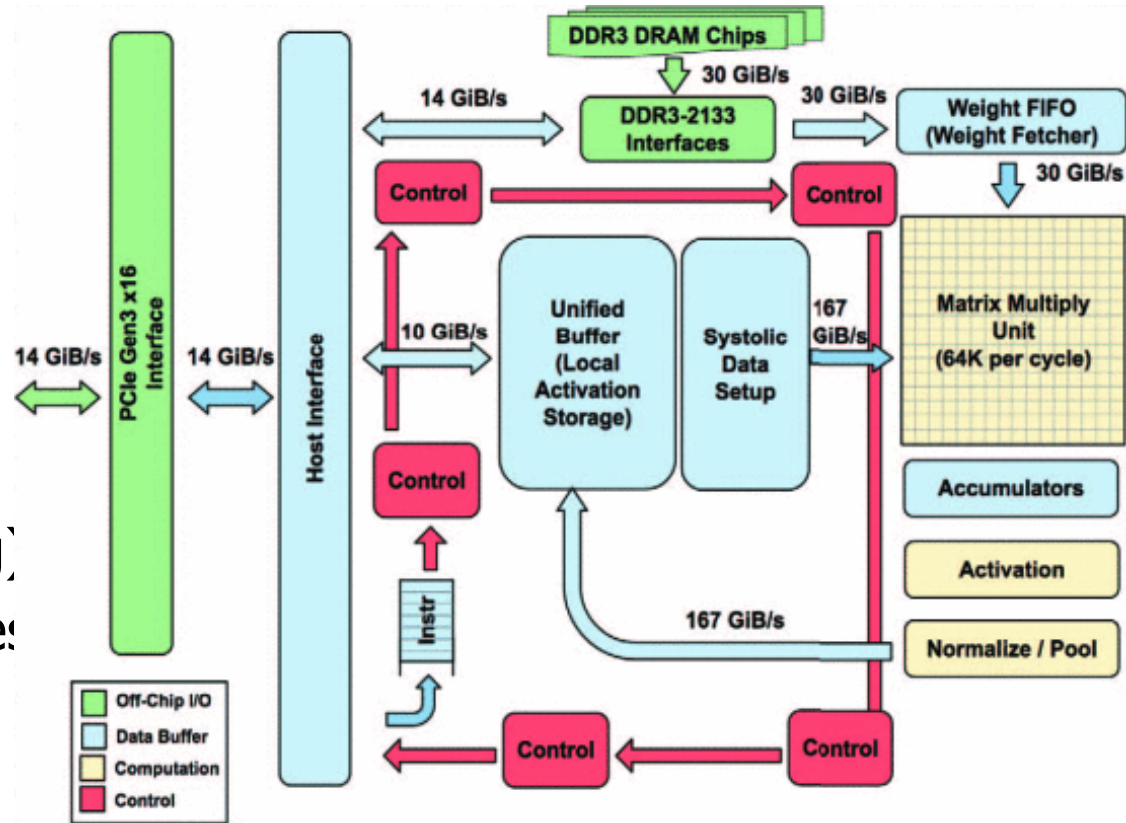
Systolic Array

- The heart of the TPU: A systolic array
- A systolic array is a network of processors that rhythmically compute and pass data through the system
- Data flows from memory in a rhythmic fashion passing through many elements before it returns to memory
- Optimized for power and area efficiency w.r.t matrix multiplication
- Trade off: limiting registers, control and operational flexibility in exchange for efficiency and much higher operation density
- A major benefit of systolic arrays is that all operand data and partial results are stored within (passing through) the processor array. There is no need to access external buses, main memory or internal caches during each operation



TPU Hardware

- Reads in 256 values per clock cycle
- Weights and Activations loaded in to Weight FIFO, reads from off chip RAM
- Matrix Unit: 64 KiB tile for weights (and another 64Kib for double buffering)
- Unified Local Buffer: Stores the outputs from matrix multiply, can be fed into matrix multiply unit again



- Activation Unit: Hardwired activation functions
- CISC instructions: Read_Host_Memory, Read_Weights, Mat.Mul./Conv., Activate, Write_Host_Memory



Concluding Remarks

- Computer Architecture is a very fast evolving field
- Go through various tech websites to be in sync with the state of the art
- “We cannot predict the future, but we can invent it” - Dennis Gabor